

Cursor 上下文编排机制详解

原创 技术人成长之路 技术人成长之路 2026年4月15日 08:31 广东

本文所称「上下文编排」即指 Cursor 的上下文管理机制：通过分层机制在合适的时机将合适的知识注入 AI 上下文。

🌟 第一部分：快速入门

📖 引言

在 AI 辅助编程中，如何向 AI 传递项目知识、编码规范和工作流程，是提升开发效率的关键。Cursor 作为领先的 AI 代码编辑器，提供了一套完整的上下文控制体系。在使用 Cursor 进行开发和项目协作时，开发者需要向 AI 发出指令或设定「规则」；Cursor 通过五种互补的机制——**AGENTS.md**、**规则 (Rules)**、**命令 (Commands)**、****技能 (Skills)** 和**子代理 (Subagents)**——帮助开发者精准编排 AI 的行为，覆盖从简单提示到复杂任务委托的丰富场景。

⚠️ 为什么需要上下文管理？

🧩 核心挑战

Cursor 的 AI Agent 每次执行任务时并不会自动记忆历史信息。大型语言模型 (LLM) 面临三个本质性限制：

1. 无状态特性

LLM 不会跨会话保留记忆，每次对话都需要重新提供项目背景

2. 上下文窗口限制

即使有较大的上下文窗口，将所有指令、规范、文档都塞入每次对话，会导致：

- **上下文污染**：关键信息被淹没在大量无关内容中
- **Token 浪费**：消耗大量 token 却没有提升输出质量
- **响应质量下降**：模型难以区分哪些信息是当前任务真正需要的

3. 跨工具协作困难

开发者常同时使用多个 AI 工具 (Cursor、Claude、Aider)，每个工具都有私有配置格式，导致文档碎片化

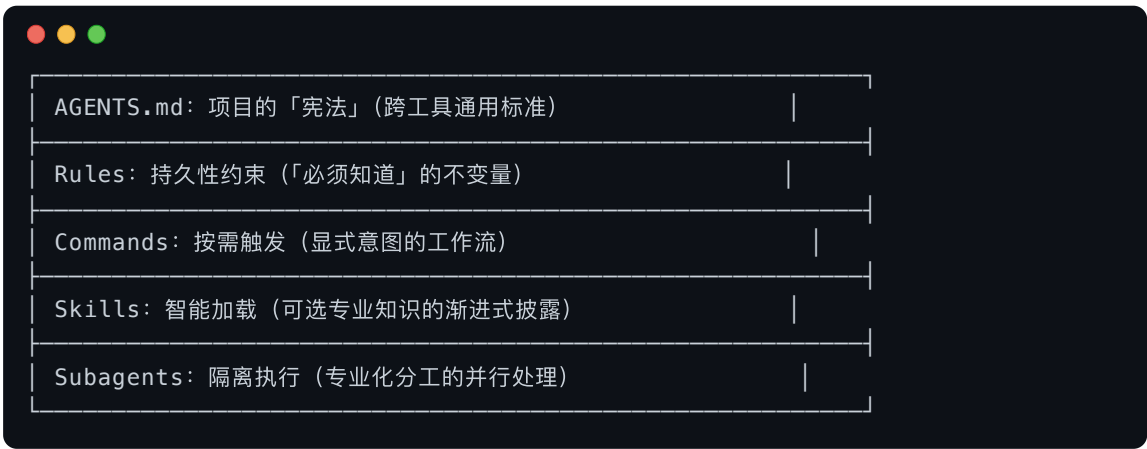
📁 解决思路：分层的上下文编排

为了解决这些问题，Cursor 提供了五种互补的机制，实现「按需供给（Just-in-Time Context）」：**在需要的时刻，将需要的知识，以需要的粒度，加载到 AI 的上下文中。

五种机制的定位与定义：

机制	定位	定义
AGENTS.md	项目级别的全局指令文件	放在项目根目录的 Markdown 文件，为 AI 提供全局指导方针
Rules	持久性的结构化指令集合	存储在 <code>.cursor/rules/</code> 的结构化指令，通过条件自动或手动附加到上下文
Commands	用户主动触发的工作流	通过 <code>/</code> 前缀手动触发的预定义工作流
Skills	动态加载的专业知识包	可选的、动态加载的专业知识包，支持脚本执行
Subagents	独立运行的专业化 AI 实例	独立运行的专业化 AI 实例，拥有独立上下文窗口

分层架构：



上图概括了五种机制的分层角色：从全局标准 → 持久约束 → 按需工作流 → 智能加载 → 隔离执行。

⚡ 五种机制对比

项目	AGENTS.md	Rules	Commands	Skills
保存位置	项目根目录	<code>.cursor/rules/</code>	<code>.cursor/commands/</code>	<code>.cursor/sk</code>
文件格式	<code>AGENTS.md</code>	<code>.mdc</code> 或 <code>.md</code>	<code>.md</code>	<code>SKILL.md</code> (文件夹内)
触发方式	始终自动适用	条件/始终/手动	仅手动（ <code>/命令名</code> ）	AI 自动判断或

项目	AGENTS.md	Rules	Commands	Skills
上下文	与父共享	与父共享	与父共享	与父共享
并行执行	不可	不可	不可	不可
再利用性	项目限定	项目限定	项目限定	可跨项目复用
主要目的	项目方针	向 AI 的持续指示	特定作业的启动	专业能力的扩
适用场合	简单的整体指示	需要条件适用的	手动执行的定型作业	通用的专业知
示例	编码规范	特定目录的规则	/code-review	GraphQL 设计的最佳实
运营重要性	推荐	强烈推荐	视需要选用	推荐

📖 第二部分：核心机制详解

📄 AGENTS.md — 项目全局指导方针

📌 定义与作用

定义：

AGENTS.md 是放置在项目根目录的 Markdown 文件，用于定义 AI 的全局指导方针和项目战略性指令。它是「为 AI 准备的 README」，是整个项目的「宪法」，也是由 Agentic AI 基金会推动的**开放标准**，旨在解决不同 AI 工具之间的配置互通问题。

作用：

- **定义项目的全局上下文**：描述技术栈、架构决策和约束
- **支持跨工具协作**：被 Cursor、Claude、Aider 等多种 AI 工具识别
- **作为所有其他指令的「基础层」**：为整个项目建立共识

特点：

- **零配置**：无需元数据，放在根目录即可生效
- **始终被加载**：类似宪法，总是在 AI 上下文中
- **应保持简洁**：只包含核心信息
- **跨工具通用**：遵循 Agentic AI 基金会开放标准
- **单体仓库对应**：可在子目录配置（如 `frontend/AGENTS.md`、`backend/AGENTS.md`）

 使用方法

文件位置： 项目根目录或其子目录中创建 `AGENTS.md`

分层架构：

层级	存储位置	作用范围与优先级	典型内容示例
全局层级	<code>~/.cursor/agents.md</code>	用户所有项目， 最低优先级	个人偏好的回复语言、 代码缩进习惯
项目根目录	<code>project-root/AGENTS.md</code>	整个代码库， 中等优先级	项目技术栈、构建工具、 全局安全策略
子目录/ 模块层级	<code>subdir/AGENTS.md</code>	仅在该子目录下生效， 最高优先级	该模块特有的设计模式、 API 调用示例、Legacy 代码处理原则

说明：全局层级路径（如 `~/.cursor/agents.md`）因工具版本可能不同，请以 Cursor 官方文档为准。

 适用场景

 适用于：

- 跨工具协作场景
- 项目技术栈声明（框架、语言、工具链）
- 核心架构决策和开发原则
- 构建命令和测试流程
- 「禁区」定义（不允许修改的文件/模块）
- 小型项目、个人项目
- 需要简单、直接、易读的指令场景

 不适用于：

- 具体的代码模板
- 频繁变化的规范
- 任务特定的详细步骤
- 需要条件判断的复杂逻辑
- 需要自动化流程或跨项目复用

 规则（Rules） — 持久可重用的系统级指令

 定义与作用

定义：

规则（Rules）是持久性的、结构化的指令集合，存储在 `.cursor/rules/` 目录中的 `.mdc`（Markdown Cursor）文件。它通过 YAML 前置元数据（Frontmatter）实现指令的动态、条件化注入，是 Cursor 内部高度精密的「地方法规」。

作用：

- **提供持久、可重用的上下文**：确保 AI 在生成代码、理解编辑时有一致的指导
- **强制执行编码标准和最佳实践**：定义 AI 在处理代码时必须遵守的不变约束
- **定义项目的「红线」和安全约束**
- **统一团队标准**：强制执行代码风格、架构决策或安全实践

特点：

- **模块化管理**：每个规则一个文件
- **支持元数据控制**：通过 `description`、`globs`、`alwaysApply` 定义何时应用、适用范围
- **灵活的适用条件**：可选择始终/特定文件/AI 判断/手动
- **文件引用**：使用 `@filename` 可引用代码
- **支持版本控制**：通过 Git 管理
- **团队规则**：可在仪表盘整体管理

 工作原理

大型语言模型在不同补全之间不会保留记忆，规则在**提示级别**提供持久、可重用的上下文。

应用后，规则内容会被加入到模型上下文的**开头**，从而在生成代码、理解编辑或协助工作流时为 AI 提供一致的指导。

 使用方法

文件位置：`.cursor/rules/*.mdc` 或 `.cursor/rules/*.md`

支持的前置元数据（Frontmatter）：

元数据字段	功能描述	对 AI 行为的影响
<code>description</code>	简要说明规则的作用	关键字段。AI 通过此描述判定该规则是否属于「Agent Requested」范畴

元数据字段	功能描述	对 AI 行为的影响
<code>globs</code>	定义适用的文件路径模式	实现「Auto Attached」。 当当前文件匹配模式时，规则自动进入上下文
<code>alwaysApply</code>	布尔值（true/false）	设置为 true 时，规则成为「始终应用」 的全局准则

四种规则类型对比：

规则类型	描述	description	alwaysApply	globs
<code>Always Apply</code>	应用于每个聊天会话	可选	<code>true</code>	留空
<code>Apply Intelligently</code>	当 Agent 根据描述判断其相关时应用	✅ 必需	<code>false</code>	留空
<code>Apply to Specific Files</code>	当文件匹配指定模式时应用	可选	<code>false</code>	✅ 必需
<code>Apply Manually</code>	在对话中被 @ 提及时应用 (例如: <code>@my-rule</code>)	可选	<code>false</code>	留空

三级规则体系：

- 1. **项目规则**：存储在 `.cursor/rules/` 中，受版本控制
- 2. **团队规则**：由团队管理员在 Cursor 仪表盘中创建和管理，适用于 Team 和 Enterprise 方案
- 3. **用户规则**：在 Cursor 设置中定义，全局生效，用于设定个人偏好

✅ 适用场景

✅ 适用于：

- 命名约定和代码风格
- 安全约束（如「永不提交 .env 文件」「不允许写硬编码密码」）
- 设计系统规范（设计 token、组件规范）
- 可访问性标准
- 数据库设计规则、API 设计模式
- 中大型团队协作项目
- 有复杂编码规范和架构要求的项目

❌ 不适用于：

- 复杂的多步骤工作流（应使用 Commands）
- 可选的专业知识（应使用 Skills）
- 需要用户确认的操作
- 一次性任务

命令（Commands） — 一键运行的工作流

📌 定义与作用

定义：

命令（Commands）是用户可以主动触发的、预定义的工作流。它存储在 `.cursor/commands/` 目录中的 `.md` 文件，通过在聊天界面输入 `/命令名` 来快速执行特定的、可重复的任务序列，代表用户的「显式意图」，提供确定性的执行结果。

作用：

- **封装可重复的工作流**：将常用或固定流程抽象成快捷命令
- **提供人体工程学的快捷方式**：减少重复输入，提高效率
- **组合多个规则和技能**：可以在命令中引用规则和技能
- **强制标准化某些常见任务指南**：确保团队按统一流程执行

特点：

- **即时执行**：可通过 `/` 调用
- **可传递参数**：支持参数化（如 `/pr 123 feature-branch`，对应 `$1`、`$2`）
- **短小精悍，易于记忆**
- **工作流定义**：可将多步骤作业总结为一个命令
- **团队命令**：可在仪表盘共享

🔧 使用方法

文件位置：`.cursor/commands/*.md`

命令类型：

1. **项目命令**：存储在 `.cursor/commands/` 中
2. **全局命令**：存储在 `~/.cursor/commands/` 中
3. **团队命令**：由团队管理员在仪表盘创建，自动同步给所有成员

参数化支持：

- `$1`、`$2`、`$3` 等表示位置参数
- `${2:-80}` 表示参数 2，默认值为 80

✅ 适用场景

✅ 适用于：

- 代码审查流程（`/code-review`）
- 组件脚手架（`/create-component Button`）
- 提交信息生成（`/commit`）
- PR 创建和描述生成（`/pr`）
- 设计转代码（`/adapt-design`）
- 发布准备、执行测试
- 安全审查
- 团队内需要频繁执行的固定流程
- 个人想要快速调用的复杂提示

❌ 不适用于：

- 总是需要应用的规则（应使用 Rules）
- 需要深度推理的任务（应使用 Subagents）
- 长时间运行的后台任务
- 应自动生效、持续适用的规则
- 由 AI 自行判断的规则与知识

🎓 技能（Skills） — 具备执行力的专业知识包

📌 定义与作用

定义：

技能（Skills）是可选的、动态加载的专业知识包，存储在 `.cursor/skills/` 目录中。它遵循 Agent Skills 开放标准（如 agentskills.io），将特定领域的知识和工作流封装成可移植、受版本控制的包，包含 SKILL.md、脚本、文档和资源；不仅提供文本指令，还可包含可执行脚本和工具。

作用：

- **提供任务特定的专业知识**：将某类专业知识打包成可复用的模块
- **包含可执行脚本和工具**：超越静态的文本指令，允许智能体运行脚本执行实际任务
- **实现上下文的按需加载**：通过「渐进式披露」在需要时才把相关知识加载进上下文
- **保持主上下文整洁**：避免上下文污染

特点：

- **渐进式披露**：从元数据到详细逐步披露信息
- **可移植**：可在多个项目复用
- **AI 根据任务自动判断适用**：通过 `description` 进行路由匹配
- **开放标准**：遵循 Agent Skills 标准格式（如 agentskills.io）
- **代码执行**：包含脚本，可高效执行 LLM 不擅长的任务

📖 渐进式披露机制

Agent Skills 采用**渐进式披露（Progressive Disclosure）**策略，分三个阶段逐步加载信息，以有效利用上下文窗口：

阶段 1：元数据路由

- 所有已安装技能的 `name` 和 `description` 都预加载到系统提示符中
- 使代理能够了解完整情况，并确定哪些技能是相关的
- 示例：用户请求"优化 React 组件性能" → AI 识别到"React Best Practices"技能相关

阶段 2：指令加载

- 如果代理确定与当前任务相关，则将 `SKILL.md` 全文加载到上下文中
- AI 了解具体指导步骤并识别任务步骤
- 示例：加载 React 性能优化的具体规则和检查清单

阶段 3：资源引用

- 当需要额外信息来执行任务时，代理会导航并发现 `SKILL.md` 中引用的其他文件、脚本和参考资料
- 示例：需要代码模板时，才读取 `templates/` 目录

这样，你就可以将大量信息整合到技能中，又不超出 Token 限制。

🔧 使用方法

文件位置：`.cursor/skills/<skill-name>/SKILL.md`

目录结构：

```
.cursor/  
├── skills/  
│   └── ui-change/
```

```
├── SKILL.md      # 技能定义（必需）
├── templates/    # 可选：代码模板
├── docs/         # 可选：详细文档
├── scripts/      # 可选：可执行脚本
└── references/   # 可选：参考文档
```

✔ 适用场景

✔ 适用于：

- UI 组件开发指南
- 特定库的最佳实践（如 TanStack、Panda CSS）
- 文档获取工具
- 调试流程和问题排查步骤
- 部署自动化、测试执行、代码生成
- API 设计、GraphQL 规范、特定框架最佳实践
- 需要智能体与开发环境或外部系统交互的自动化任务

✘ 不适用于：

- 必须始终遵守的约束（应使用 Rules）
- 简单的快捷命令（应使用 Commands）
- 需要完全隔离上下文的任务（应使用 Subagents）

🤖 子代理（Subagents） — 支持分布式并行处理的专业化 AI 助手

📌 定义与作用

定义：

子代理（Subagents）是独立运行的专业化 AI 实例，拥有自己的上下文窗口和系统提示。它存储在 `.cursor/agents/` 目录中的 `.md` 文件。Cursor 的代理可以委托任务给专门的辅助代理，每个子代理在自己的上下文窗口中运作，处理特定类型的作业，并将结果返回给父代理。

作用：

- **执行需要专门化的任务**：处理特定类型的作业
- **提供独立的上下文空间**：避免污染主对话
- **支持并行处理**：多个子代理可以同时执行不同任务

特点：

- **上下文隔离**：子代理拥有独立上下文，不影响主对话

- **并行处理**：支持多个子代理同时执行不同任务
- **专业定制**：可为子代理配置领域专用提示、工具和模型
- **可复用**：子代理可在多个项目中重复使用

 上下文隔离原理

传统的单代理模型在面对「重构大量文件以适配新的 API 规范」等任务时，会因对话历史过长导致逻辑混乱。子代理实现了**上下文隔离 (Context Isolation)**：

- **主代理作为协调者**：负责制定全局计划并分发
- **每个子代理拥有全新的上下文窗口**：只接收与其特定任务相关的背景数据
- **子代理从空的上下文开始**：无法访问之前的对话历史，因此父代理需要在提示中包含相关信息
- **支持并行执行**：多个子代理可以同时工作（例如一个在研究旧代码逻辑，另一个在编写新测试，第三个在实现功能），互不干扰

 使用方法

文件位置：`.cursor/agents/<agent-name>.md`

两种运行模式：

模式	动作	适合的场合
Foreground	阻塞处理直到子代理完成，一旦完成立即返回结果	需要按顺序获得输出的任务
Background	立即返回控制给调用方，子代理独立运作	长时间执行任务或并行处理的作业流

使用方式：在 Cursor 中使用 `@agent-name` 提及代理，或由主代理根据任务复杂度自动派生。

内置子代理

内置子代理	职责范围	运行机制
Explore	深入研究代码库	使用更轻量、快速的模型，在极短时间内扫描大量文件并返回总结
Terminal	执行与解释命令结果	专门处理复杂的 shell 交互，捕获报错并尝试自动修复
Parallel	并行执行代码编辑	在隔离环境中同时修改多个不相关的模块

✅ 适用场景

✅ 适用于：

- 规划代理（生成详细实施计划）
- 探索代理（快速搜索代码库）
- 审查代理（使用强大模型进行代码审查）
- 研究代理（联网查找文档和解决方案）
- 复杂代码审查、安全检测
- 大规模重构、深度 Bug 追踪
- 需要隔离与专业分工的场景

❌ 不适用于：

- 简单的格式化任务
- 可以用 Skill 解决的问题
- 不需要隔离上下文的操作

🚀 第三部分：实践指南

📁 推荐项目结构

```
项目根目录/
├── AGENTS.md                # 项目整体的基本方针
├── frontend/AGENTS.md      # 前端特有的指示
├── backend/AGENTS.md       # 后端特有的指示
├── .cursor/
│   ├── rules/
│   │   ├── api-design.mdc  # API 设计模式
│   │   ├── database-schema.mdc # DB 设计规则
│   │   └── deployment-flow.mdc # 部署流程
│   ├── commands/
│   │   ├── code-review.md  # 代码审查执行
│   │   ├── create-pr.md    # PR 创建
│   │   └── run-tests.md     # 测试执行
│   ├── skills/
│   │   ├── react-best-practices/ # React 最佳实践
│   │   ├── graphql-best-practices/ # GraphQL 知识
│   │   │   └── SKILL.md
│   │   ├── kubernetes-ops/      # K8s 运营知识
│   │   │   └── SKILL.md
│   │   ├── accessibility/      # 无障碍性
│   │   │   └── SKILL.md
│   └── agents/
│       ├── verifier.md         # 验证子代理
│       └── security-reviewer.md # 安全审查子代理
```

💡 实战示例

📁 场景：编写 React 项目的操作指南

1. 模式1：AGENTS.md（简易版）

```
● ● ●
# Project Instructions

## React 组件开发
- 使用 TypeScript
- 优先函数型组件
- Props 必须进行类型定义
```

适用于：小型项目和简单说明

2. 模式2：规则（Rules）

`.cursor/rules/react-components.mdc`：

```
● ● ●
---
description: "React 组件开发的规则"
globs: ["src/components/**/*.tsx"]
alwaysApply: false
---
# React 组件规则
## 组件创建时
- 必须定义 Props 接口
- 避免默认 export
- 参考 @component-template.tsx
```

适用于：应用于特定目录的规则

3. 模式3：命令（Commands）

`.cursor/commands/create-component.md`：

```
● ● ●
# /create-component
# 步骤

1. 创建组件文件
2. 定义 Props 接口
3. 创建测试文件
```

使用方法：`/create-component Button`

4. 模式4：技能（Skills）

`.cursor/skills/react-best-practices/SKILL.md` :

```
---
name: React Best Practices
description: React 组件开发的最佳实践。重视性能优化、重渲染防止、Hooks 的适当使用。
---
# React Best Practices 技能
## 何时使用
- 创建或修正 React 组件时
- 需要性能优化时
- 使用 Hooks 时
- 解决重渲染问题时
## 指示
### 组件设计
- 自定义 Hook 使用 `use` 前缀
- Props 接口必须进行类型定义
- 组件遵循单一责任原则
### 性能优化
- `useMemo` 和 `useCallback` 仅在必要时使用
- `useEffect` 的依赖数组必须明确指定
- 对于大型列表，考虑使用虚拟化
### 重渲染防止
- `memo` 仅在必要时使用（避免过度优化）
- Context 的值适当进行 Memoization
- 识别不必要重渲染的原因
```

适用于： 自动适用行业标准的最佳实践，并在代码审查时加以利用

5. 模式5：子代理 (Subagents)

`.cursor/agents/verifier.md` :

```
---
name: verifier
description: 验证已完成的工作，确认实现是否正常运作，并执行测试
---
# Verifier 子代理
此子代理验证已完成的工作，确认实现是否正常运作，执行测试，并报告成功和未完成的部分。
## 验证步骤
1. 确认已实现的代码
2. 执行单元测试
3. 执行集成测试
4. 检查错误或警告
5. 报告结果
```

适用于： 需要独立验证、执行测试且不希望污染主对话的场景；或当你想要独立验证某些事情而不污染主要背景时

安全审查示例 `.cursor/agents/security-reviewer.md` :



```
---
name: security-reviewer
description: 检查代码中的注入、XSS、硬编码秘密等常见漏洞
---

# Security Reviewer 子代理
您是安全专家。执行代码的安全审查，识别潜在漏洞。
## 检查项目
1. SQL 注入
2. XSS（跨站脚本攻击）
3. 硬编码秘密
4. 认证和授权问题
5. 遵守安全的编码实践
```

⚠️ 常见陷阱与解决方案

陷阱	症状	解决方案
百科全书式规则	规则文件数千行，AI 很少真正使用	拆分为多个小规则文件（20—100 行），使用 <code>globs</code> 或 <code>description</code> 实现智能应
命令爆炸	有 30+ 个命令，记不住	只保留 5—10 个常用命令，其余转为技能
技能路由失败	AI 从不加载某个技能	重写 <code>description</code> ，使用任务中会出现的关键词
过度使用子代理	简单任务也启动子代理，响应慢	先尝试技能，只在真正需要隔离或并行时用子代理
上下文污染	每次对话都很慢，Token 消耗高	检查 <code>alwaysApply: true</code> 的规则，减少到最小集（3—5 个核心规则）
规则冲突	不同规则或 AGENTS.md 与 Rules 给出矛盾指令	明确规则优先级（如 Rules > AGENTS.md），使用 <code>globs</code> 限制作用域
AGENTS.md 过于冗长	文件超过 500 行	提取详细规范到 Rules，只保留核心概念
命令参数传递错误	参数没有正确传递	检查参数语法（ <code>\$1</code> 、 <code>\$2</code> ），确保在命令中正确引用
技能描述模糊	AI 不知道何时加载技能	使用具体的动词和场景描述，如「Use when creating API endpoints」
子代理上下文丢失	子代理不了解必要背景	父代理在调用时必须包含所有必要上下文
跨工具不兼容	切换工具时需要重新配置	优先使用开放标准（AGENTS.md、Agent Skills），通过符号链接保持兼容

🎯 第四部分：总结与核心原则

📄 本篇小结

Cursor 通过 **AGENTS.md**、**Rules**、**Commands**、**Skills**、**Subagents** 五种机制，实现**「按需供给 (Just-in-Time Context)」**的上下文编排：在需要的时刻，将需要的知识以需要的粒度加载到 AI 上下文中。这样既能避免无状态、上下文窗口和跨工具协作带来的困境，又能减少上下文污染与 Token 浪费。合理分层使用这五种机制，可将 Cursor 从「聊天机器人」升级为得力的「智能开发伙伴」，实现从提示词到生产的高效工作流。

🕒 机制选择一览表

下表与第一部分「五种机制对比」互补：前者侧重技术属性，本表侧重选型与场景。

机制	按场景选择	核心定位	建议
AGENTS.md	全局规范	项目的「宪法」	少量、简单项目规范直接用
Rules	持久约束	必须始终遵守的约束	长期、结构化规则用 Rules
Commands	重复任务	明确的、可重复的工作流	经常重复的流程用 Commands
Skills	可含脚本执行 / 专业知识	可选的、特定任务的指导	复杂领域知识或流程用 Skills
Subagents	复杂任务 / 隔离或并行	需要隔离或深度推理的任务	需要并行或独立上下文处理用 Subagents

若以上都不匹配，可考虑一次性提示词。

📌 核心原则

实践中可参考以下五项原则：

1. 🏠 分层管理原则

- **基础用 AGENTS.md**：项目的「宪法」
- **不变量用 Rules**：必须始终遵守的约束
- **意图用 Commands**：明确的、可重复的工作流
- **专业知识用 Skills**：可选的、特定任务的指导
- **专家用 Subagents**：需要隔离或深度推理的任务

2. 👁️ 渐进式披露原则

不要将所有信息一次性加载到上下文中。通过元数据路由、条件应用和懒加载机制，确保 AI 只看到当前任务真正需要的信息。

3. 🌐 开放标准优先原则

优先使用 AGENTS.md 和 Agent Skills 等开放标准，以实现跨工具兼容。对于 Cursor 专有功能（如 Rules、Subagents），确保其与开放标准配合良好。

4. 🔄 持续优化原则

- 当 AI 犯同样错误 3 次时，就该写规则或技能了
- 当某个提示词被重复使用 5 次以上时，就该封装成命令
- 当命令变得过于复杂（超过 50 行）时，考虑转换为技能
- 当任务需要深度推理或并行处理时，引入子代理

5. 👥 团队协作原则

- 将所有配置纳入版本控制
- 为每个 Rule、Command 和 Skill 编写清晰的说明
- 定期审查和清理冗余内容
- 使用团队规则和命令强制执行组织标准

以上原则与机制选择一览表可作为日常选型与复盘时的速查参考。

🔗 参考资源

- Cursor 官方文档 – Rules
- Cursor 官方文档 – Commands
- Cursor 官方文档 – Skills
- Cursor 官方文档 – Subagents
- Agent Skills 开放标准
- Agentic AI 基金会 – AGENTS.md 标准

👍 喜欢作者

“ 您的鼓励是我创造的最大动力 ”

收录于  Cursor

